⚛ REACT

BASICS
- JS library for building UIs (Facebook). View layer only.
- Virtual DOM → diffs changes → updates only changed nodes
- SPA-focused, component-based, unidirectional data flow

COMMANDS
```
npx create-react-app myapp  // create project
cd myapp && npm start  // dev server :3000
npm run build  // production bundle
```

FOLDER STRUCTURE
- public/index.html ← page template ( )
- src/index.js ← JS entry point
- src/App.js ← root component

JSX
- JavaScript XML — HTML-like syntax in JS
- Must return ONE parent element (wrap in or <>)
- className not class; htmlFor not for
- JS expressions inside { } curly braces
- Self-close empty tags: <input /> <img />

```
const el = <h1>Hello {name}!</h1>;
const el = <input type='text' />;
const list = (<ul><li>A</li><li>B</li></ul>);
// Expression: {5 + 5}, {isTrue ? 'Yes':'No'}
```

RENDERING (v18)
```
// index.js
import ReactDOM from 'react-dom/client';
const root = ReactDOM.createRoot(
  document.getElementById('root'));
root.render(<App />);
```

FUNCTIONAL COMPONENT
- Plain JS function returning JSX. Modern preferred.

```
function Greet(props) {
  return <h2>Hello, {props.name}!</h2>;
}
// Arrow function style:
const Greet = (props) => <h2>{props.name}</h2>;
export default Greet;
```

CLASS COMPONENT
- ES6 class, extends React.Component, has render()
- Use when: need state, lifecycle, event logic (legacy)

```
class Car extends React.Component {
  constructor(props) {
    super(props);  // REQUIRED
    this.state = { color: 'red' };
  }
  render() {
    return <h2>Car: {this.state.color}</h2>;
  }
}
```

PROPS
- Read-only data passed parent → child via attributes
- Access: props.name (functional) / this.props.name (class)
- Props are IMMUTABLE — cannot be changed by child

```
// Parent:
<Greeting name='Ali' age={21} />
// Child (functional):
function Greeting({ name, age }) {
  return <p>{name} is {age} yrs old</p>;
}
```

STATE + useState ⚡IMPORTANT
- State = internal data managed by component
- State change → React re-renders only that component
- useState returns [value, setter] pair
- NEVER mutate state directly → always use setter

```
import { useState } from 'react';
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={()=>setCount(count+1)}>
        +1</button>
    </div>
  );
}
```

EVENTS
- camelCase: onClick, onChange, onSubmit, onMouseOver
- Handler in { } — pass reference, don't call it

```
function shoot() { alert('Goal!'); }
<button onClick={shoot}>Shoot</button>
// Inline arrow:
<button onClick={() => alert('Hi')}>Click</button>
// Input handling:
const [val, setVal] = useState('');
<input onChange={e => setVal(e.target.value)}
  value={val} />
```

FULL EXAMPLE: Counter App
```
import { useState } from 'react';
function App() {
  const [n, setN] = useState(0);
  return (
    <>
      <h1>Count: {n}</h1>
      <button onClick={()=>setN(n+1)}>+</button>
      <button onClick={()=>setN(n-1)}>-</button>
      <button onClick={()=>setN(0)}>Reset</button>
    </>
  );
}
export default App;
```

COMPONENT LIFECYCLE (Class)
- Mounting: constructor() → render() → componentDidMount()
- Updating: render() → componentDidUpdate()
- Unmount: componentWillUnmount()

useEffect (Functional Lifecycle)
```
import { useEffect } from 'react';
useEffect(() => {
  // runs after render (componentDidMount)
  fetch('/api').then(r=>r.json()).then(setData);
  return () => { /* cleanup */ };
}, []);  // [] = run once only
```

MULTIPLE STATE VARS
```
const [name, setName] = useState('');
const [age, setAge]   = useState(0);
const [on, setOn]     = useState(false);
```

CONDITIONAL RENDERING
- Show/hide components based on state/props

```
// Ternary operator:
{isLoggedIn ? <Dashboard/> : <Login/>}
// && short-circuit:
{count > 0 && <p>Items: {count}</p>}
// Return null to hide:
function Warn({show}) {
  if (!show) return null;
  return <p>Warning!</p>;
}
// Full login/logout example:
function App() {
  const [logged, setLogged] = useState(false);
  return (
    <div>
      {logged
        ? <button onClick={()=>setLogged(false)}>
            Logout</button>
        : <button onClick={()=>setLogged(true)}>
            Login</button>
      }
      {logged && <h1>Welcome Back!</h1>}
    </div>
  );
}
```

MAP FUNCTION + LISTS + KEYS
- map() transforms each array element → new array
- Keys help React track which items changed/added/removed
- Keys must be unique among siblings

```
// Basic map:
const nums = [1,2,3,4,5];
const doubled = nums.map(n => n * 2);
// [2,4,6,8,10]

// Rendering a list:
const fruits = ['Apple','Banana','Cherry'];
<ul>
  {fruits.map((f,i) => <li key={i}>{f}</li>)}
</ul>

// With objects (use unique id as key):
const users = [
  {id:1, name:'Ali'}, {id:2, name:'Sara'}];
<ul>
  {users.map(u =>
    <li key={u.id}>{u.name}</li>)}
</ul>

// Map inside component:
function UserList({users}) {
  return (
    <div>
      {users.map(u => (
        <div key={u.id}>
          <h3>{u.name}</h3>
          <p>{u.email}</p>
        </div>
      ))}
    </div>
  );
}
```

PASSING CALLBACK (Child→Parent)
```
// Parent:
function Parent() {
  const [msg, setMsg] = useState('');
  return <Child onSend={setMsg} />;
}
// Child:
function Child({ onSend }) {
  return <button onClick={()=>onSend('Hi')}>
    Send</button>;
}
```

REACT ROUTER (basics)
```
npm install react-router-dom
import {BrowserRouter as Router,
  Route, Routes, Link} from 'react-router-dom';
<Router>
  <Link to='/about'>About</Link>
  <Routes>
    <Route path='/' element={<Home/>} />
    <Route path='/about' element={<About/>}/>
  </Routes>
</Router>
```

FETCH DATA (useEffect + fetch)
```
const [data, setData] = useState([]);
useEffect(() => {
  fetch('https://api.example.com/users')
    .then(r => r.json())
    .then(d => setData(d));
}, []);
```

CSS IN REACT
```
// Inline:
<h1 style={{color:'red', fontSize:'20px'}}>Hi</h1>
// Object:
const s = {color:'blue', padding:'10px'};
<div style={s}>Box</div>
// className:
<p className='my-class'>Text</p>
```

A ANGULAR

BASICS
- Google-maintained open-source SPA framework
- TypeScript-based (compiled to JS)
- Full framework: routing, forms, HTTP built-in
- MEAN stack: MongoDB, Express, Angular, Node

CLI COMMANDS
```
npm install -g @angular/cli  // install CLI
ng new myapp  // create project
cd myapp
ng serve --open  // run dev server
ng generate component mycomp
ng build  // production build
```

TYPESCRIPT BASICS
- Superset of JS with static types
- Compiled to JS via tsc

```
// Types:
let name: string = 'Ali';
let age: number = 21;
let active: boolean = true;
let sizes: string[] = ['S','M','L'];
enum Color {Red, Green, Blue}

// Function with types:
function greet(n: string): string {
  return 'Hello ' + n;
}

// Interface:
interface User {
  name: string;  age: number;
}
```

- Access: public(default), private, protected, readonly

```
// Parameter property shortcut:
class Foo {
  constructor(public x:number, private y:string){}
}
// Generics:
function compare<T>(a:T, b:T): number {...}
```

@COMPONENT DECORATOR
- Metadata that marks class as Angular component

```
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'My App';
}
```

NGMODULE
- Root module that bootstraps the app

```
@NgModule({
  declarations: [AppComponent],  // components
  imports: [BrowserModule],  // Angular mods
  bootstrap: [AppComponent]  // root comp
})
export class AppModule {}
```

DATA BINDING
- Interpolation: {{ title }} → display value
- Property: [src]='imageUrl' → bind property
- Event: (click)='fn()' → bind event
- Two-way: [(ngModel)]='name' → sync model

DECORATORS
- @Component — marks a component class
- @NgModule — marks a module class
- @Injectable — marks a service class
- @Input / @Output — pass data between comps
- Similar to Java annotations; add metadata

DIRECTIVES
```
*ngFor="let item of items; let i=index"  // loop
*ngIf="condition; else elseBlock"  // cond
<ng-template #elseBlock><p>Else</p></ng-template>
[ngSwitch]="val" *ngSwitchCase="1"
[ngStyle]="{color:'red', background:'yellow'}"
[ngClass]="{active:true, hide:false}"  // cond CSS
```

PIPES (| OPERATOR)
- Transform data in templates before display
- Chainable with | (pipe operator)

```
{{ 'hello' | uppercase }}  // HELLO
{{ 'HELLO' | lowercase }}  // hello
{{ birthday | date:'M/d/yyyy' }}  // 4/15/1988
{{ birthday | date:'fullDate' }}  // Friday, Apr...
{{ price | currency:'USD' }}  // $9.99
{{ price | currency:'PKR' }}  // PKR9.99
{{ 0.259 | percent }}  // 26%
{{ 3.14159 | number:'1.2-2' }}  // 3.14
// Chaining pipes:
{{ amount | currency:'USD' | uppercase }}
// JSON pipe (debugging):
{{ myObject | json }}
```

PARENT→CHILD & CHILD→PARENT
```
// Child component:
@Input() entries: Entry[];
// Parent template:
<app-child [entries]='data'></app-child>

@Output() added = new EventEmitter<Entry>();
this.added.emit(newEntry);  // child emits
// Parent template:
<app-child (added)='onAdd($event)'></app-child>
```

ROUTING
```
const routes: Routes = [
  {path:'about', component:AboutComponent},
  {path:'**', component:NotFoundComponent}  // 404
];
<a routerLink='about'>About</a>  // no reload
<router-outlet></router-outlet>  // renders here
this.router.navigate(['block2']);  // programmatic
```

routerLink=History API (no reload), href=full reload.

SERVICES & DI
```
@Injectable() class DataService{...}
```

Register: providers:[DataService] in @NgModule.
Inject: constructor(private svc: DataService)

HTTP & PIPES
```
import {HttpClientModule} from
  '@angular/common/http';  // import in module
this.http.get('/api/data')  // GET
this.http.post('/api/data', body)  // POST
```

Proxy: proxy.conf.json → ng serve --proxy-config proxy.conf.json

REACT FORMS (Controlled)
- Controlled: state = source of truth, value={state}
- onChange → setState on every keystroke. Recommended.

```
function MyForm() {
  const [name,setName] = useState('');
  const [msg,setMsg] = useState('');
  const handleSubmit = (e) => {
    e.preventDefault();  // stop reload
    alert('Name:'+name+' Msg:'+msg);
  };
  return (
    <form onSubmit={handleSubmit}>
      <input type='text' value={name}
        onChange={e=>
          setName(e.target.value)}/>
      <textarea value={msg}
        onChange={e=>
          setMsg(e.target.value)}/>
      <input type='submit'/>
    </form>
  );
}
```

NODE.JS

BASICS
- JS runtime environment (uses Chrome V8 engine)
- Server-side JS — no DOM; has File I/O, HTTP, DBs
- Non-blocking, event-driven, async I/O
- 'N' in MEAN/MERN stack

CLIENT vs SERVER JS
```
Client (Browser):     Server (Node):
- Has DOM, window   - Has File I/O, DBs
- fetch() to call APIs  - Handles HTTP requests
- Runs in browser    - Runs in terminal
- No file access       - No DOM access
```

EVENT LOOP
- Single-threaded; uses async callbacks
- Non-blocking: no waiting on I/O → high concurrency
- Queue → stack → execute → callback → repeat

NPM BASICS
```
npm init  // create package.json
npm init -y  // auto-yes defaults
npm install express  // install package
npm install -g nodemon  // install globally
npm start  // run start script
node app.js  // run file directly
```

SIMPLE HTTP SERVER (built-in)
```
const http = require('http');
const server = http.createServer(
  (req, res) => {
    res.writeHead(200,
      {'Content-Type':'text/plain'});
    res.end('Hello World!');
  }
);
server.listen(3000);
// → http://localhost:3000
```

EXPRESS.JS (Web Framework)
```
npm install express

const express = require('express');
const app = express();

// GET route:
app.get('/', (req, res) => {
  res.send('Home Page');
});

// POST route:
app.post('/data', (req, res) => {
  res.json({ status: 'ok' });
});

app.listen(3000, () => {
  console.log('Server on port 3000');
});
```

ROUTING
```
app.get('/path', handler);  // GET request
app.post('/path', handler);  // POST request
app.all('/secret', handler);  // all HTTP methods
```

EXPRESS ROUTER (modularize)
```
// users.js
const router = express.Router();
router.get('/', listUsers);
router.post('/', addUser);
router.get('/:id', getUser);
router.delete('/:id', deleteUser);
module.exports = router;

// app.js
app.use('/users', require('./users'));  // mount
```

REQUEST & RESPONSE
Request: req.query (GET params ?a=10), req.params (route params /:id), req.body (POST form/JSON)
```
// GET /add?a=10&b=20 → req.query.a, req.query.b
// GET /add/:a/:b → req.params.a, req.params.b
// POST body → req.body.a  // needs middleware
```
Response: res.send(), res.json(), res.status(404), res.redirect('/'), res.end()
```
res.status(200).json({result: 30});
```

MIDDLEWARE
- function(req, res, next) — runs before route handler
- Call next() to pass to next middleware. Order matters!

```
app.use((req,res,next) => {
  console.log(req.method, req.url);
  next();  // pass control to next
});
app.use(express.json());  // parse JSON body
app.use(express.urlencoded({extended:true}));
```
Error middleware: (err,req,res,next) — 4 args. next(err) skips to error handler.

REQUIRE / MODULES
```
// Built-in modules:
const http = require('http');
const fs   = require('fs');
const path = require('path');

// NPM package:
const express = require('express');

// Own file:
const myMod = require('./mymodule');
```

ASYNC / CALLBACKS
```
// Callback style:
fs.readFile('f.txt', (err, data) => {
  if (err) throw err;
  console.log(data.toString());
});

// Promise / async-await:
async function getData() {
  const res = await fetch(url);
  const json = await res.json();
  return json;
}
```

MVC STRUCTURE
/public = static files, /routes = controllers
/views = templates (.hbs), /bin = executables
layout.hbs = master page, child view = {{{body}}}
Environment vars: process.env.PORT || 3000
dotenv: require('dotenv').config(); .env file

JWT AUTHENTICATION
```
const jwt = require('jsonwebtoken');
jwt.sign(payload, secret, options);  // create token
```
Auth middleware: extract JWT from Authorization header, verify, attach user to req. 401 if invalid.
```
app.use(passport.initialize());
app.use('/api/', passport.authenticate(
  'jwt', {session:false, failWithError:true}));
```

MONGODB

NoSQL BASICS
- Not Relational — no fixed schema, no SQL
- Designed for large, unstructured datasets
- NoSQL types: Key-Value, Column, Graph, Document
- MongoDB = Document database (most popular NoSQL)
- 'M' in MEAN/MERN stack

STRUCTURE
```
Database
  └── Collection  (like a SQL table)
      └── Document  (like a SQL row)
          └── Field: Value pairs (JSON)
```

DOCUMENT / BSON
- Documents stored as BSON (Binary JSON)
- BSON adds more types: dates, ObjectId, binary
- _id field auto-added; unique primary key (ObjectId)

```
{
  _id: ObjectId('...'),
  first_name: 'John',
  age: 20,
  address: {
    city: 'Lahore', country: 'PK'
  },
  tags: ['student','dev']
}
```

CRUD OPERATIONS
- INSERT — add documents

```
// Insert one:
db.users.insertOne({
  name: 'Maria', age: 30
});

// Insert many:
db.users.insertMany([
  {name:'Ali'}, {name:'Sara'}
]);
```
- FIND — query documents

```
db.users.find()  // all docs
db.users.find({age:30})  // filter
db.users.findOne({name:'Ali'})

// Comparison operators:
db.users.find({age: {$gt: 18}})  // >
db.users.find({age: {$lt: 30}})  // <
db.users.find({age: {$gte: 18}})  // >=
db.users.find({age: {$lte: 30}})  // <=
db.users.find({age: {$ne: 25}})  // !=

// Multiple conditions (AND):
db.users.find({
  status:'A', age:{$lt:30}
})

// Projection (select fields):
db.users.find({}).project(
  {name:1, _id:0}  // only name, no _id
)
```
- UPDATE — modify documents

```
// Update one:
db.users.updateOne(
  {name: 'Ali'},  // filter
  {$set: {age: 25}}  // update
);

// Update many:
db.users.updateMany(
  {status:'P'},
  {$set: {status:'A'}}
);
```
- DELETE — remove documents

```
db.users.deleteOne({name:'Ali'});
db.users.deleteMany({age:{$lt:18}});
db.users.drop();  // delete collection
db.dropDatabase();  // delete entire DB
```

MONGO SHELL COMMANDS
```
show dbs  // list databases
use mydb  // switch/create db
show collections  // list collections
db  // current db
```

SQL vs MONGODB TERMS
```
SQL          →  MongoDB
Table        →  Collection
Row          →  Document
Column       →  Field
Primary Key  →  _id
JOIN         →  $lookup (difficult)
SELECT       →  find()
INSERT INTO  →  insertOne/Many()
UPDATE       →  updateOne/Many()
DELETE       →  deleteOne/Many()
```

AGGREGATION ($lookup = JOIN)
```
db.collection('Article').aggregate([
  { $lookup: {
      from: 'User',  // join with
      localField: 'Author',  // my field
      foreignField: '_id',  // their field
      as: 'user'  // output array
  }},
  { $unwind: '$user' },  // flatten array
  { $project: {
      _id:0, Title:1,
      Author_Name: '$user.Name'
  }}
]);
```

INDEXING
- Indexes improve query performance
- Without index → full collection scan

```
db.users.createIndex({lastName:1});  // asc
db.users.createIndex(
  {email:1}, {unique:true});
db.articles.createIndex({tags:1});
```

NODE.JS + MONGODB CONNECTION
```
npm install mongodb

const {MongoClient} = require('mongodb');
const uri = 'mongodb://localhost:27017';
const client = new MongoClient(uri);

async function run() {
  await client.connect();
  const db = client.db('mydb');
  const col = db.collection('users');

  // Insert:
  await col.insertOne({name:'Zara'});

  // Find:
  const all = await col.find({}).toArray();

  // Update:
  await col.updateOne(
    {name:'Zara'},
    {$set:{age:22}}
  );

  // Delete:
  await col.deleteOne({name:'Zara'});
  await client.close();
}
run();
```

EXPRESS + MONGODB (Full CRUD API)
```
const express = require('express');
const {MongoClient, ObjectId} =
  require('mongodb');
const app = express();
app.use(express.json());  // parse JSON

let db;  // global db reference
MongoClient.connect(
  'mongodb://localhost:27017'
).then(client => {
  db = client.db('myapp');
  app.listen(3000);
});

// GET all users:
app.get('/api/users', async(req,res)=>{
  const users = await
    db.collection('users')
    .find({}).toArray();
  res.json(users);
});

// POST create user:
app.post('/api/users', async(req,res)=>{
  const result = await
    db.collection('users')
    .insertOne(req.body);
  res.status(201).json(result);
});

// PUT update user:
app.put('/api/users/:id',
  async(req,res)=>{
  await db.collection('users')
    .updateOne(
      {_id: new ObjectId(req.params.id)},
      {$set: req.body}
    );
  res.json({msg:'Updated'});
});

// DELETE user:
app.delete('/api/users/:id',
  async(req,res)=>{
  await db.collection('users')
    .deleteOne(
      {_id: new ObjectId(req.params.id)}
    );
  res.json({msg:'Deleted'});
});
```

MONGOOSE (ODM for MongoDB)
```
npm install mongoose
const mongoose = require('mongoose');
mongoose.connect(
  'mongodb://localhost:27017/myapp');

// Define schema + model:
const userSchema = new mongoose.Schema({
  name: {type:String, required:true},
  age: Number,
  email: {type:String, unique:true}
});
const User = mongoose.model('User',
  userSchema);

// CRUD with Mongoose:
const u = new User({name:'Ali',age:21});
await u.save();  // insert
await User.find({});  // find all
await User.findById(id);  // find by id
await User.updateOne(
  {name:'Ali'}, {$set:{age:22}});
await User.deleteOne({name:'Ali'});
```

QUICK RECALL
- $set — update specific field
- $gt $lt $gte $lte $ne — comparison ops
- $in — match array of values
- $push — add to array, $pull — remove from array
- $and $or $not — logical operators
- ObjectId — 12-byte unique document identifier
- Embedded doc — nested object inside doc (16MB max)
- Controlled(React)=state+onChange, Uncontrolled=ref
- @Input=[brackets] parent→child, @Output=(parens)
- Callback→Promise(.then)→async/await (evolution)